Table 1: Comparison of Docker and LXC

	Docker	LXC
Level	Application	Operating system
Consists of	Microservices and the portability of applications across environments in mind.	Like a lightweight virtual machine, every LXC has its own file system, networking, users, and processes.
Aim	Creates a more user-friendly platform for container orchestration and deployment by abstracting LXC or other container runtimes under its engine.	Aims to create environments that closely mimic a fully functional Linux system.
Isolation	It isolates apps at the process level using container technology (cgroups, namespaces). Unlike other container operating systems that run multiple services or an entire Linux OS, Docker's containers are more strictly sandboxed to run a single application or process.	Uses Linux namespaces (PID, NET, IPC, etc) to provide isolation for processes, filesystems, and networks. Within the container, it can run several services and complete Linux distributions. Although the degree of isolation is still shared with the host kernel, it is closer to traditional virtualisation.
Deployment	Docker is best suited for packaging, deploying, and operating single applications with all their dependencies in isolated environments because it was primarily created for application-level containerisation. Strong tools from Docker, such as Kubernetes, Docker Compose, and Docker Swarm, are available for managing numerous containers in intricate deployments.	When users need to run several programs and services (like a web server, database, etc) inside one container to simulate managing an entire system, LXC is better suited for system-level containerisation.
Setup	It provides a simpler command-line interface.	More complex than Docker. In LXC, system-level operations and configuration files require more manual configuration for networking, storage, and security management and configuration.
Image management	Contains a sophisticated image management system.	Does not have a central image repository.
Orchestration	The most popular container runtime in Kubernetes, Docker offers native support for container orchestration via Docker Swarm.	There are no sophisticated orchestration features in LXC itself. Scripts can be used to manage and configure LXC containers, but no orchestration tools are included with the system.
Security	Compared to LXC, Docker's default configuration is often more secure right out of the box because it includes tools like Notary for image signing, which makes sure containers only run trusted code.	Linux kernel features like namespaces and cgroups can be used to secure LXC, but further effort may be needed to properly isolate containers from the host.

LXC components

Given below are the basic components of LXC we need to start with.

If we want to create a container, the following command is required:

```
lxc-create -n  testcontainer -t Ubuntu
```

To start a container, we need the following code:

```
lxc-start -n testcontainer
```

To attach to a running container and execute commands inside it, type:

```
lxc-attach -n testcontainer
```

The command to stop the container is:

```
lxc-stop -n testcontainer
```

To destroy the containers and delete all the data, type:

```
lxc-destroy -n testcontainer
```

Advantages of LXC

Efficacy: Unlike virtual machines, which need a hypervisor and different OS instances, containers use the host OS's kernel.

Performance: LXCs offer almost native speed due to the lack of overhead associated with executing different operating systems, such as virtual machines.